

Accelerating Large-Scale Distributed Training via Chunk-Wise Gradient Sparsification and Pipelined Communication

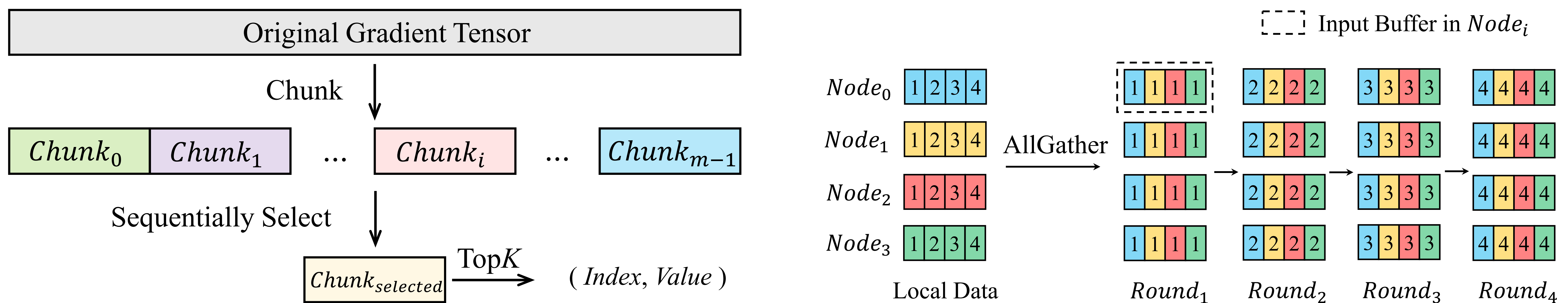
Xian-Yu Zhu, Ruo-Han Wu, Jun-Shi Chen*, and Hong An

School of Computer Science and Technology, University of Science and Technology of China; Laoshan Laboratory

E-mail: zhuxy@mail.ustc.edu.cn Journal of Computer Science and Technology, 2026

1. Abstract

Gradient communication is a major bottleneck in large-scale data-parallel training. Existing TopK sparsification reduces transmitted data but still scans the full gradient and produces sparse data that are difficult to reduce efficiently. We propose CHUNK-GRADCOMP, a framework that combines chunk-wise gradient selection with pipelined sparse communication. CHUNK-TOPK selects important values from one gradient chunk per iteration while carrying the remaining values as error-compensation residuals. CHUNK-ALLGATHER splits sparse data into segments and overlaps communication with local reduction; AUTO PD selects the pipeline depth using hardware-aware performance models. On a 32-GPU H800 cluster, the combined method reduces communication overhead by nearly 80% and achieves $3.12\text{--}4.74\times$ speedup over DenseAllReduce at 0.1% compression.



2. Key Contributions

1. CHUNK-TOPK reduces the per-iteration selection scope from $O(n)$ to $O(n/m)$ and retains omitted gradients through error compensation.
2. CHUNK-ALLGATHER divides sparse data into segments and overlaps communication with local gradient reduction, reducing buffer cost from $O(NK)$ to $O(NK/M)$.
3. AUTO PD uses lightweight performance models to select the pipeline depth, while SRAG provides a ring-based alternative when the depth equals the number of workers.

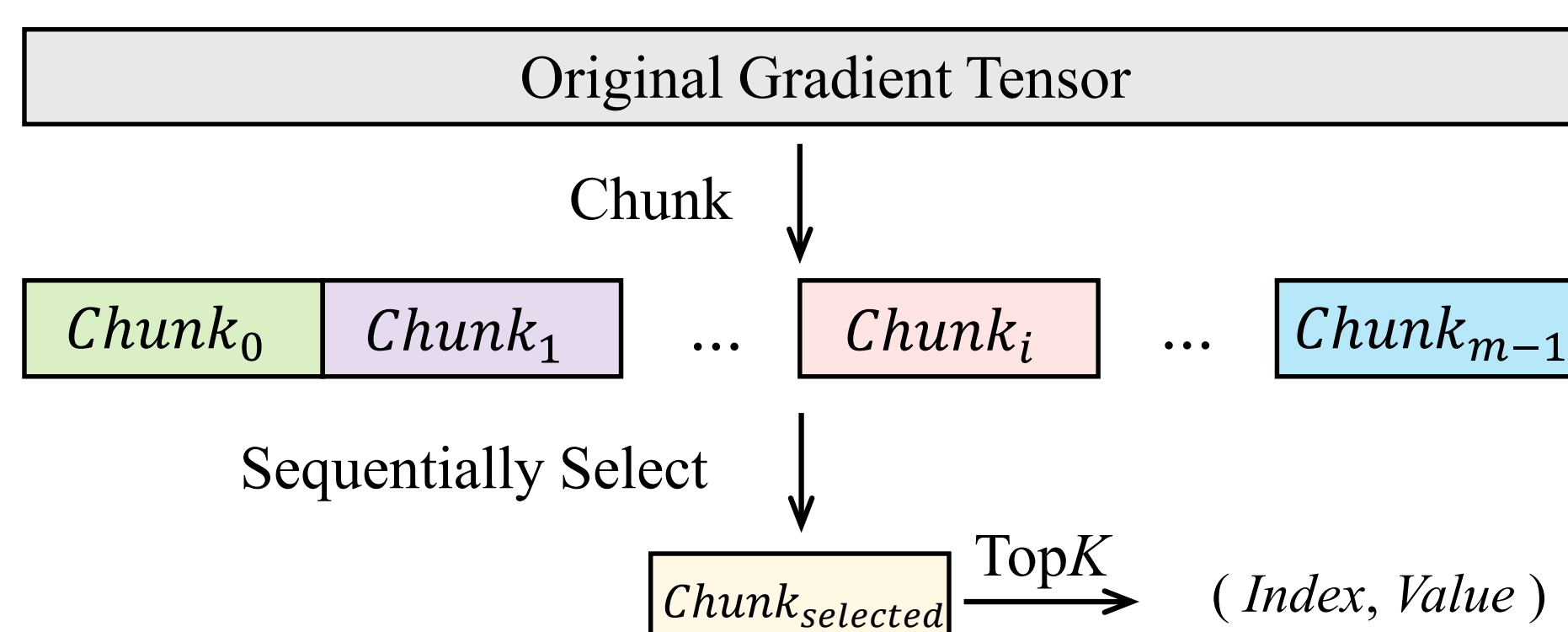
3. CHUNK-TOPK

Design motivation. The design is guided by two established observations:

(1) *error compensation* shows that gradients omitted in one iteration remain useful when accumulated and reintroduced later;

(2) *asynchronous training* demonstrates that gradient updates can tolerate moderate delays without preventing effective convergence. Therefore, it is unnecessary to inspect the entire gradient at every iteration.

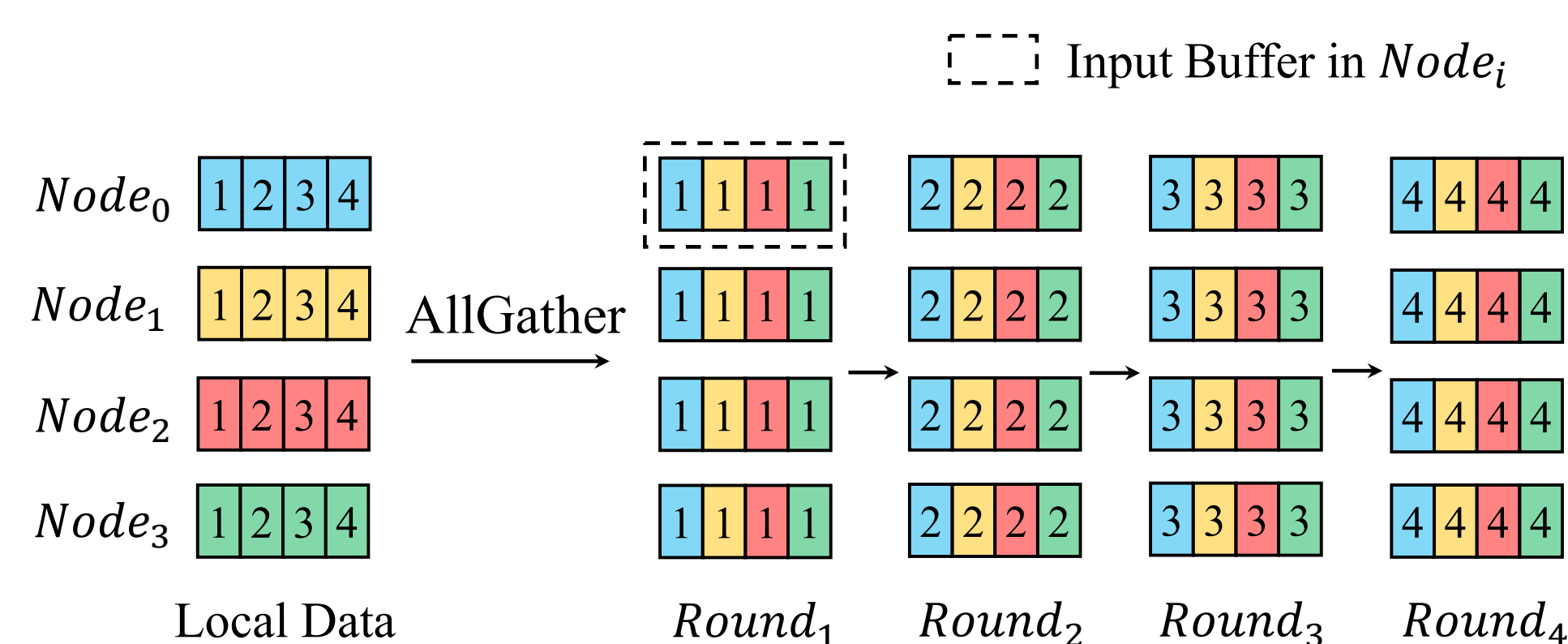
Core idea. Partition the gradient tensor into m equal-sized chunks. At iteration t , apply TopK only to chunk c_t and transmit its largest-magnitude values. The other chunks and unselected values are accumulated in a residual buffer and revisited in later iterations.



Why it works. Sequentially revisiting all chunks prevents information from being permanently discarded. Moderate chunk counts preserve accuracy close to full-gradient TopK while substantially reducing selection cost.

4. CHUNK-ALLGATHER and AUTO PD

Pipelined sparse communication. After sparsification, each worker uses AllGather followed by local index-add reduction. CHUNK-ALLGATHER divides one communication round into M segments so that communication of segment i overlaps with reduction of segment $i - 1$.



Double buffering allows a received segment to be reduced immediately. AutoPD estimates total time and selects the depth with minimum predicted time. The best predictors achieve 8.29% MAPE for AllGather and 1.46% for index add.

Experimental evidence. On four HGX H800 nodes (32 GPUs), Chunk-AllGather is fastest across tested compression ratios. The full method achieves $3.12\text{--}4.74\times$ end-to-end speedup at 0.1% compression and $3.00\text{--}4.41\times$ at 1% relative to DenseAllReduce.

